

# Pràctica 2: Introducció a Prolog

Lògica en la Informàtica

FIB

Antoni Lozano

Q1 2025–2026

# Objectius

Aquesta pràctica té com a objectius:

- Tenir un primer **contacte amb Prolog**, un llenguatge **declaratiu** on es descriu
  - **quina** és l'estructura del problema amb **regles** i **fets** i
  - **quin** és l'objectiu mitjançant **consultes**

a diferència dels llenguatges imperatius que indiquen **com** arribar a la solució.

- Arribar a escriure **programes senzills** en Prolog.

# Referències

Com a guia d'estudi teniu

- El document **p6.pdf**, que conté les notes de classe d'IL sobre fonaments de la programació lògica
- El document **p6sols.pl**, amb solucions d'exercicis del document anterior (dins l'avís del Racó *Col·lecció d'apunts bàsics de lògica*)
- Els exemples de programes Prolog de l'enunciat de la pràctica
- El fitxer **codi.pl**, que conté el codi d'aquestes transparències
- La documentació completa de SWI Prolog de la pàgina oficial (<https://www.swi-prolog.org/>)

# Introducció a Prolog

1 Inici

2 Sintaxi

3 Objectius

4 Llistes

5 Aritmètica

6 Operador de tall

# Orígens

- El nom prové de *Programming in logic*
- Desenvolupat a la **Universitat Aix-Marseille** (60s–70s)
- Prolog és hereu de la recerca en sistemes de **deducció automàtica**
- Utilitzat en els projectes **ESPRIT** (Europa) i **Fifth Generation** (Japó, amb la intenció de crear ordinadors intel·ligents)
- És utilitzat en àmbits com la **IA** o el **llenguatge natural**
- És un dels principals llenguatges de programació lògica, juntament amb **ASP** i **Datalog**

# Ús de l'interpret

Per executar l'interpret, escriuiu en un terminal:

```
> swipl
Welcome to SWI-Prolog (threaded, 64 bits, version 8.4.3)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free
  software.
Please run ?- license. for legal details.

For online help and background,
visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

?-
```

# Compilació

Els programes s'haurien de desar en fitxers amb extensió `pl`.

Si tenim un fitxer `codi.pl`, escrivim dins l'interpret

```
?- [codi].
```

i ja podrem fer consultes sobre tot allò que s'ha declarat a `codi.pl`.

Per acabar la sessió amb l'interpret, escriviu

```
?- halt.
```

# Exemple

Programar en Prolog vol dir **descriure** el problema que tenim entre mans.  
Es fa ús d'un programa **consultant** la descripció que s'ha fet del problema.

La manera més senzilla de descriure un problema és enumerant **fets**.

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).
```

Ara podem fer consultes.

```
?- mes_gros(ruc,gos).  
true.
```

```
?- mes_gros(gat,elefant).  
false.
```

# Exemple

Programar en Prolog vol dir **descriure** el problema que tenim entre mans.  
Es fa ús d'un programa **consultant** la descripció que s'ha fet del problema.

La manera més senzilla de descriure un problema és enumerant **fets**.

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).
```

Ara podem fer consultes.

```
?- mes_gros(ruc,gos).  
true.
```

```
?- mes_gros(gat,elefant).  
false.
```

# Exemple

Programar en Prolog vol dir **descriure** el problema que tenim entre mans.  
Es fa ús d'un programa **consultant** la descripció que s'ha fet del problema.

La manera més senzilla de descriure un problema és enumerant **fets**.

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).
```

Però si preguntem a l'inrevés, també respon **false**:

```
?- mes_gros(elefant,gat).  
false.
```

## Exemple

Programar en Prolog vol dir **descriure** el problema que tenim entre mans. Es fa ús d'un programa **consultant** la descripció que s'ha fet del problema.

La manera més senzilla de descriure un problema és enumerant **fets**.

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).
```

Per resoldre-ho, tenim dues opcions: ampliar el conjunt amb tots els fets possibles o afegir una **regla** general, que podria ser:

```
mes_gros(X,Y) :- mes_gros(X,Z), mes_gros(Z,Y).
```

La regla estableix que `mes_gros(X,Y)` és cert si existeix un `Z` que compleix `mes_gros(X,Z)` i `mes_gros(Z,Y)`.

# Exemple

Ara tenim el conjunt de fets i regles:

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).  
mes_gros(X,Y) :- mes_gros(X,Z), mes_gros(Z,Y).
```

Com volíem, tenim

```
?- mes_gros(elefant,gat).  
true .
```

Però si preguntem a l'inrevés

```
?- mes_gros(gat,elefant).  
ERROR: Stack limit (1.0Gb) exceeded
```

En la secció 3 veurem per què l'interpret actua així.

# Exemple

Ara tenim el conjunt de fets i regles:

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).  
mes_gros(X,Y) :- mes_gros(X,Z), mes_gros(Z,Y).
```

Com volíem, tenim

```
?- mes_gros(elefant,gat).  
true .
```

Però si preguntem a l'inrevés

```
?- mes_gros(gat,elefant).  
ERROR: Stack limit (1.0Gb) exceeded
```

En la secció 3 veurem per què l'interpret actua així.

## Exemple

Una solució que evita la recursió infinita és reservar la recursivitat per un nou predicat:

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).  
mGros(X,Y) :- mes_gros(X,Y).  
mGros(X,Y) :- mes_gros(X,Z), mGros(Z,Y).
```

Ara respon correctament i proporciona diferents solucions (prement espai o punt i coma):

```
?- mGros(X,gat).  
X = ruc;  
X = elefant;  
X = cavall;  
false.
```

# Introducció a Prolog

1 Inici

2 **Sintaxi**

3 Objectius

4 Llistes

5 Aritmètica

6 Operador de tall

# Termes i predicats

## Termes

Un **terme** és:

- Un **àtom**: gat, a, fXY, r1, una\_altra\_ronda, 'I això'
- Un **nombre**: 42, -666
- Una **variable**: X, Rinoceront, \_, \_234
- Un **terme compost**: mes\_gros(X,gos), f(g(X,\_),5)

Es pot comprovar quina mena de terme és una cadena de símbols X amb els predicats `atom(X)`, `number(X)`, `var(X)` i `compound(X)`.

## Predicats

Un **predicat** és un àtom o un terme compost.

# Termes i predicats

## Exemple

El nostre programa conté predicats aïllats o relacionats:

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).  
mGros(X,Y) :- mes_gros(X,Y).  
mGros(X,Y) :- mes_gros(X,Z), mGros(Z,Y).
```

Ara veurem com es defineix formalment un programa a partir dels predicats.

# Clàusules, programes i consultes

Hem començat parlant de **fets** i **regles**.

## Fets

Un **fet** és un predicat seguit de punt.

```
mes_gros(elefant,cavall).
```

## Regles

Una **regla** és un **cap** seguit d'un signe **:-**, un **cos** i un punt, on:

- el **cap** és un predicat i
- el **cos** és una seqüència de predicats separats per comes.

```
mGros(X,Y) :- mes_gros(X,Z), mGros(Z,Y).
```

# Clàusules, programes i consultes

## Programes

Els fets i les regles s'anomenen **clàusules**. Un **programa** és una seqüència de clàusules.

## Exemple

L'argument clàssic

*Tots els homes són mortals*

*Sòcrates és un home*

---

*Per tant, Sòcrates és mortal*

es tradueix a Prolog com

```
mortal(X) :- home(X). % regla  
home(socrates).      % fet
```

# Clàusules, programes i consultes

## Consultes

Una **consulta** té la mateixa estructura que el cos d'una regla: una seqüència de predicats separats per comes i acabada en punt.

## Exemple

Donat el programa següent, es poden fer consultes.

```
mortal(X) :- home(X).  
home(socrates).
```

```
?- mortal(X).  
X = socrates.
```

```
?- home(plato).  
false.
```

```
?- mortal(socrates), mortal(plato).  
false.
```

# Introducció a Prolog

- 1 Inici
- 2 Sintaxi
- 3 Objectius**
- 4 Llistes
- 5 Aritmètica
- 6 Operador de tall

# Predicats predefinitos

Alguns predicats predefinitos són:

- `true`, que sempre és cert
- `false` (o `fail`), que sempre és fals
- `=(X,Y)` és cert si i només si  $X$  i  $Y$  es poden unificar  
Per més comoditat, `=(X,Y)` es pot escriure  $X = Y$
- `\=(X,Y)` és cert si i només si `=(X,Y)` és fals  
Es pot escriure com  $X \neq Y$
- `\+ X`, que és cert quan  $X$  no es pot demostrar
- `write(X)`, que escriu el terme  $X$  i sempre és cert; en el cas que  $X$  sigui una variable, escriu el seu valor
- `n\`, que salta de línia i sempre és cert

# Unificació

El concepte d'unificació és fonamental per entendre com es processen les consultes en Prolog.

## Unificació

Dos termes **es poden unificar** si són idèntics o bé es poden fer idèntics mitjançant la instanciació de variables.

## Exemples

```
?- 17 = N.
```

```
N = 17.
```

```
?- mGros(X, gat) = mGros(cavall, gat).
```

```
X = cavall.
```

# Unificació

La mateixa variable s'ha d'instanciar amb el mateix valor en una expressió o una consulta.

?-  $f(X, g(X), a) = f(a, Y, Z)$ .

$X = Z, Z = a,$

$Y = g(a)$ .

?-  $p(X, 1) = p(2, Y), X = Y$ .

**false.**

L'excepció és la variable anònima `_` perquè cada ocurrència representa una variable diferent.

?-  $p(_, 2, 2) = p(1, Y, _)$ .

$Y = 2$ .

# Execució d'objectius

Fer una consulta vol dir demanar a Prolog que provi de demostrar que els predicats inclosos en la consulta es poden fer certs amb les instanciacions necessàries.

La cerca d'aquesta demostració se'n diu **execució d'objectius**.

# Execució d'objectius

## Exemple

Quan fem la consulta `mortal(socrates)` al programa

```
mortal(X) :- home(X).  
home(socrates).
```

Prolog respon que és cert després de seguir els passos:

- 1 `mortal(socrates)` és l'**objectiu inicial**
- 2 Prolog intenta unificar l'objectiu amb el primer fet o cap de regla que troba, que és `mortal(X)` i **s'instancia**  $X = \text{socrates}$
- 3 S'estén la instanciació al cos de la regla, `home(socrates)`, que serà l'**objectiu actual**
- 4 Prolog intenta unificar l'objectiu actual amb algun fet o regla i ho fa amb el fet `home(socrates)`; **l'objectiu actual té èxit**
- 5 Això vol dir que **l'objectiu inicial també té èxit**

# Execució d'objectius

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).  
mes_gros(X,Y) :- mes_gros(X,Z), mes_gros(Z,Y).
```

La consulta `mes_gros(gat,elefant)` porta la pila al límit de recursió.

- 1 Examina totes les clàusules **en ordre**.
- 2 Com que no la troba com a fet, la unifica amb `mes_gros(X,Y)` **instanciant** X a gat i Y a elefant
- 3 Per demostrar `mes_gros(gat,elefant)` llença els **subobjectius** `mes_gros(gat,Z)` i `mes_gros(Z,elefant)` per a algun Z
- 4 Per demostrar `mes_gros(gat,Z)` l'**unifica** de nou amb el terme `mes_gros(X,Y)` de l'última regla, que s'aplica indefinidament.

# Execució d'objectius

```
mes_gros(elefant,cavall).
mes_gros(cavall,ruc).
mes_gros(ruc,gos).
mes_gros(ruc,gat).
mes_gros(X,Y) :- mes_gros(X,Z), mes_gros(Z,Y).
```

La consulta `mes_gros(elefant,gat)`, en canvi, retorna **true**.

- ❶ La unifica amb `mes_gros(X,Y)` **instanciant** X a `elefant` i Y a `gat`
- ❷ Per demostrar `mes_gros(elefant,gat)` llença els **subobjectius** `mes_gros(elefant,Z)` i `mes_gros(Z,gat)` per a algun Z
- ❸ El **primer** subobjectiu s'unifica amb el terme `mes_gros(elefant,cavall)` instanciant Z a `cavall`
- ❹ El **segon** subobjectiu és, doncs, `mes_gros(cavall,gat)`, que s'unifica amb `mes_gros(X,Y)` en l'última regla
- ❺ **Nous subobjectius**; `mes_gros(cavall,Z)` i `mes_gros(Z,gat)`. La instanciació Z = `ruc` retorna el resultat **true**

# Execució d'objectius

## Exercici

Donat el programa

```
mes_gros(elefant,cavall).  
mes_gros(cavall,ruc).  
mes_gros(ruc,gos).  
mes_gros(ruc,gat).  
mGros(X,Y) :- mes_gros(X,Y).  
mGros(X,Y) :- mes_gros(X,Z), mGros(Z,Y).
```

Expliqueu per què la consulta

```
?- mGros(X,gat).
```

produeix com a resultat

```
X = ruc;  
X = elefant;  
X = cavall;  
false.
```

# Introducció a Prolog

- 1 Inici
- 2 Sintaxi
- 3 Objectius
- 4 Llistes**
- 5 Aritmètica
- 6 Operador de tall

# Notació

Una seqüència de termes escrits entre parèntesis quadrats és una **llista**.

Per exemple,

[elefant, cavall, ruc, gos, gat]

és una llista formada per cinc àtoms. Però els elements d'una llista també poden ser llistes:

[a, b, [], f(a), g(f(a,X)), [a,b,c], X]

## Llista buida

La **llista buida** es representa amb [].

# Notació

El primer element d'una llista és el **cap** i la resta d'elements és la **cua**.  
Per exemple, en la llista

$$L = [a, b, [], f(a), g(f(a,X)), [a,b,c], X]$$

tenim que

**cap**: a

**cua**:  $[b, [], f(a), g(f(a,X)), [a,b,c], X]$

Una **variant de la notació** permet fer referència al cap i la cua d'una llista.

Si el cap de L és X i la cua és Y, llavors podem escriure L amb

$$[X \mid Y]$$

En l'exemple anterior,  $L = [X \mid Y]$  on

$X = a$

$Y = [b, [], f(a), g(f(a,X)), [a,b,c], X]$

# Notació

El primer element d'una llista és el **cap** i la resta d'elements és la **cua**.  
Per exemple, en la llista

$$L = [a, b, [], f(a), g(f(a,X)), [a,b,c], X]$$

tenim que

**cap**: a

**cua**:  $[b, [], f(a), g(f(a,X)), [a,b,c], X]$

Una **variant de la notació** permet fer referència al cap i la cua d'una llista.

Si el cap de L és X i la cua és Y, llavors podem escriure L amb

$$[X \mid Y]$$

En l'exemple anterior,  $L = [X \mid Y]$  on

$X = a$

$Y = [b, [], f(a), g(f(a,X)), [a,b,c], X]$

# Notació

## Exemple

La interacció següent amb l'interpret

```
?- [] = [X | Y].
```

**false.**

confirma que una cua buida no té cap i, per tant, X no es pot unificar amb res.

En canvi, la consulta

```
?- [a] = [X | Y].
```

```
X = a,
```

```
Y = [].
```

té èxit i confirma que una llista formada per un element té com a cap aquell element i com a cua la llista buida.

# Notació

Però la notació de llistes és encara més versàtil. Podem descriure un **nombre constant d'elements** abans de la barra vertical. Per exemple,

?- [1,2,3,4,5] = [X, Y, Z | T].

X = 1,

Y = 2,

Z = 3,

T = [4, 5].

Exemple: segon element d'una llista

Fent ús d'aquesta notació, podem definir el predicat

`segon([_,X | _],X).`

per obtenir el segon element d'una llista, com en la consulta

?- segon([a, b, c, d, e], S).

S = b.

# Notació

Però la notació de llistes és encara més versàtil. Podem descriure un **nombre constant d'elements** abans de la barra vertical. Per exemple,

?- [1,2,3,4,5] = [X, Y, Z | T].

X = 1,

Y = 2,

Z = 3,

T = [4, 5].

## Exemple: segon element d'una llista

Fent ús d'aquesta notació, podem definir el predicat

`segon([_,X | _],X).`

per obtenir el segon element d'una llista, com en la consulta

?- `segon([a, b, c, d, e], S).`

S = b.

# Exemples

## Pertinença a una llista

`pert(X, L) : X pertany a la llista L`

Predicat predefinit: `member/2`

(la notació `/2` indica que `member` té dos arguments)

```
pert(X, [X | _]).
```

```
pert(X, [_ | L]) :- pert(X, L).
```

## Observació

Un element pertany a una llista si:

- és el primer de la llista o
- pertany a la resta de la llista

Recursió en el segon argument.

# Exemples

## Subconjunts

subconjunt(S,C) : S és un subconjunt de C, on  
S i C són llistes sense repeticions

Predicat predefinit: subset/2

```
subconjunt([], _).
subconjunt([X | R], T) :-
    pert(X, T),
    subconjunt(R, T).
```

## Observacions

En notació de conjunts:

- $\emptyset$  és subconjunt de qualsevol conjunt
- $\{X\} \cup R \subseteq T$  si  $X \in T$  i  $R \subseteq T$

Recursió en el primer argument.

# Exemples

## Concatenació de llistes

`conc(L1, L2, L3)` : L3 és la concatenació de L1 i L2

Predicat predefinit: `append/3`

```
conc([], L, L).
```

```
conc([X | L1], L2, [X | L3]) :- conc(L1, L2, L3).
```

## Observació

La concatenació d'una llista que comença per l'element X amb una altra llista ha de tenir X com a primer element (i la resta conserva l'ordre original).

Recursió en el primer argument.

# Exemples

## Concatenació de llistes

`conc(L1, L2, L3)` : L3 és la concatenació de L1 i L2

Predicat predefinit: `append/3`

`conc([], L, L).`

`conc([X | L1], L2, [X | L3]) :- conc(L1, L2, L3).`

## Exemple d'ús

Amb la consulta `conc([1], [2,3], L)`, l'interpret fa el següent:

- ❶ Unifica amb el cap de la 2a clàusula:  $X=1$ ,  $L1=[]$ ,  $L2=[2,3]$
- ❷ Llença el subobjectiu `conc([], [2,3], L3)`
- ❸ El subobjectiu unifica amb la 1a clàusula:  $L3 = [2,3]$
- ❹ El 3r arg. del predicat consultat és, doncs,  $L = [X|L3] = [1,2,3]$

# Exemples

## Últim d'una llista

`ultim(L, X)`: X és l'últim element de L

Predicat predefinit: `last/2`

```
ultim(L, X) :- conc(_, [X], L).
```

## Observació

L'element X és l'últim de la llista L si L és la concatenació d'alguna llista amb la llista [X].

# Exemples

## Llista permutada

`permutacio(L, P)`: P és una permutació de la llista L

Predicat predefinit: `permutation/2`

```
permutacio([], []).  
permutacio([X | L], P) :-  
    permutacio(L, Q),  
    conc(Q1, Q2, Q),  
    conc(Q1, [X | Q2], P).
```

## Observació

Es pot obtenir una permutació de `[X | L]` inserint X en una posició qualsevol d'una permutació de L.

Recursió en tots dos arguments.

# Exemples

## Consulta

Es poden obtenir totes les permutacions de [a, b, c] amb

```
?- permutacio([a, b, c], S), write(S), nl, false.
```

```
[a, b, c]
```

```
[b, a, c]
```

```
[b, c, a]
```

```
[a, c, b]
```

```
[c, a, b]
```

```
[c, b, a]
```

```
false.
```

També prement l'espai o el punt i coma després de la consulta:

```
?- permutacio([a, b, c], S).
```

```
S = [a, b, c] ;
```

```
S = [b, a, c] ;
```

```
...
```

# Introducció a Prolog

- 1 Inici
- 2 Sintaxi
- 3 Objectius
- 4 Llistes
- 5 Aritmètica**
- 6 Operador de tall

# Operadors

L'ús de l'aritmètica pot produir comportaments inesperats.

Podem comprovar l'exemple següent:

```
?- 7 + 4 = 11.
```

```
false.
```

Cal tenir en compte que, en Prolog, la igualtat serveix per **unificar termes** (fa un *pattern matching*), però no els avalua encara que siguin expressions aritmètiques.

Això explica les consultes:

```
?- 7 + 4 = N + 4.
```

```
N = 7.
```

```
?- 1 + 3 = 3 + 1.
```

```
false.
```

# Operadors

Per avaluar una expressió aritmètica, cal dir-ho explícitament a Prolog. Una manera de fer-ho és fer ús de l'operador `is`. Si `X` és una variable, la consulta

```
?- X is 7 + 4.  
X = 11.
```

força l'avaluació de l'expressió que té a la dreta i la unifica amb la variable que té a l'esquerra (`X`).

Cal tenir en compte:

- Que **no funciona a l'inrevés**:

```
?- 7 + 4 is X.
```

```
ERROR: Arguments are not sufficiently instantiated
```

- L'expressió de la dreta **ha de poder-se avaluar**. En concret, totes les variables de l'expressió han d'estar instanciades

# Operadors

Altres operadors que permeten avaluar expressions aritmètiques:

- $>$  (més gran)
- $<$  (més petit)
- $=<$  (més petit o igual)
- $>=$  (més gran o igual)
- $=\backslash=$  (no igual)
- $=:=$  (igual)

# Exemples

## Mida

`mida(L,N)`: la mida de `L` és `N`

Predicat predefinit: `length/2`

```
mida([], 0).
```

```
mida([_ | L], N):- mida(L, N1), N is N1 + 1.
```

## Observació

Fem `N is N1 + 1` per obtenir el resultat numèric en `N`  
(el predicat `mida(L, N1 + 1)` no seria correcte).

# Exemples

## Factorial

`fact(N, F) : F és el factorial de N`

`fact(0, 1):- !.`

`fact(N, F):- N1 is N - 1, fact(N1, F1), F is N * F1.`

## Observacions

- Fem `N1 is N - 1` perquè `fact(N1, F1)` necessita `N1` instanciat
- Fem `F is N * F1` per forçar l'avaluació de `N * F1`

# Exemples

```
% xifres(L,N) escriu les maneres d'obtenir N a partir de +, -, *
% dels elements de la llista L
```

```
% exemple:
```

```
% ?- xifres([4,9,8,7,100,4], 380).
```

```
% ... 4 * (100-7) + 8 ... <-----
```

```
% ... ((100-9) + 4) * 4
```

```
% ... ..
```

```
xifres(L,N):-
```

```
... subconjunt(L,S), ... % S = [4,8,7,100]
```

```
... permutation(S,P), ... % P = [4,100,7,8]
```

```
... expressio(P,E), ... % E = 4 * (100-7) + 8
```

```
... N is E,
```

```
... write(E), nl, false.
```

```
% E = ( 4 * (100-7) ) + 8
```

```
% ..... +
```

```
% ..... / \
```

```
% ..... * ..... 8
```

```
% ..... / \
```

```
% ..... 4 -
```

```
% ..... / \
```

```
% ..... 100 - 7
```

# Exemples

```
expressio([X],X).
```

```
expressio(L, E1 + E2):-
```

|   |   |   |  |                       |
|---|---|---|--|-----------------------|
| → | → | → |  | • append(L1, L2, L),  |
| → | → | → |  | • L1 \= [], L2 \= [], |
| → | → | → |  | • expressio(L1, E1),  |
| → | → | → |  | • expressio(L2, E2).  |

```
expressio(L, E1 - E2):-
```

|   |   |   |  |                       |
|---|---|---|--|-----------------------|
| → | → | → |  | • append(L1, L2, L),  |
| → | → | → |  | • L1 \= [], L2 \= [], |
| → | → | → |  | • expressio(L1, E1),  |
| → | → | → |  | • expressio(L2, E2).  |

```
expressio(L, E1 * E2):-
```

|   |   |   |  |                       |
|---|---|---|--|-----------------------|
| → | → | → |  | • append(L1, L2, L),  |
| → | → | → |  | • L1 \= [], L2 \= [], |
| → | → | → |  | • expressio(L1, E1),  |
| → | → | → |  | • expressio(L2, E2).  |

% Com afegir la divisió entera?

# Introducció a Prolog

- 1 Inici
- 2 Sintaxi
- 3 Objectius
- 4 Llistes
- 5 Aritmètica
- 6 Operador de tall**

Hi ha un operador especial anomenat **operador de tall** (!) que permet dirigir l'execució d'objectius en Prolog i, en particular, el *backtracking*.

- L'operador ! sempre **té èxit**
- Un cop satisfet, en la crida actual al predicat que conté el tall, **impedeix**
  - el *backtracking* abans del tall i
  - l'aplicació de les següents regles amb al mateix predicat com a cap.

## Funcionament del tall

Abans d'arribar al tall,  $a_1$ ,  $a_2$ , ... poden fer *backtracking*.

```
p :- a1, a2, ..., !, b1, b2, ...
p :- c1, c2, ...
```

Després d'arribar al tall,

- no es farà *backtracking* en  $a_1$ ,  $a_2$ , ...
- no s'aplicarà la segona regla (ni, per tant, els objectius  $c_1$ ,  $c_2$ , ...)

# Exemples

% Ja hem vist aplicacions de l'operador de tall, com ara:

```
fact(0,1):-!.  
fact(N,F):-N1 is N-1, fact(N1,F1), F is N * F1.
```

% Provem a eliminar-lo i consultar

% fact(0,F), write(F), false.

# Exemples

```
% El tall elimina l'exploració posterior de l'objectiu
% però també el backtracking dels subobjectius anteriors de l'objectiu
% Per exemple:
```

```
p(1).
p(2).
q(a).
q(b).
r(3,4,5).
r(X,Y,Z):- p(X), q(Y), !, s(Z).
```

```
% el tall ! treu de la pila les alternatives per a p(), q() i r()
```

```
r(5,6,7).
s(3).
s(4).
```

```
h(X,Y,Z):- r(X,Y,Z).
h(a,b,c).
```

```
%% ?- h(A,B,C), write([A,B,C]), nl, false.
%% [3,4,5]
%% [1,a,3]
%% [1,a,4]
%% [a,b,c]
%% false.
```

# Exemples

```
% EXEMPLE DE L'OPERADOR DE TALL: INTERVAL
% Escriure un predicat interval que generi tots els enters entre
% una fita inferior (1r argument) i una fita superior (2n argument).
% El resultat ha de ser una llista d'enters (3r argument).
% Si la fita inferior és més gran que la superior, retornar [].
% Exemples:
% ?- interval(3,11,X).
% X = [3,4,5,6,7,8,9,10,11]
%
% ?- interval(7,4,X).
% X = []
```

```
interval(A,B,[]) :-
→   B < A, !.
```

```
interval(A,A,[A]) :- !.
```

```
interval(A,B,[A|X]) :-
→   C is A + 1,
→   interval(C,B,X).
```

```
rang(A,B) :-
→   interval(A,B,X),
→   write(X).
```

```
% Provar a treure l'operador de tall
% Provar amb interval i amb rang
```

# Exemples

```

|
% der(E,V,D) == "la derivada de E respecte de V és D"

der(X,X,1):-var(X),!.
% ! es fa servir per aturar la cerca de solucions
der(C,_,0):-
|   number(C).
der(A+B,X,U+V):-
|   der(A,X,U),
|   der(B,X,V).
der(A*B,X,A*V+B*U):-
|   der(A,X,U),
|   der(B,X,V).

```

# Operadors lògics

## Conjunció

$\text{and}(A, B) :- A, B.$

( $\text{and}(A, B)$  és cert si A i B són certs.)

## Disjunció

Dues solucions. En tots dos casos,  $\text{or}(A, B)$  és cert si A o B són certs.

1.

$\text{or}(A, B) :- A.$

$\text{or}(A, B) :- B.$

2.

$\text{or}(A, B) :- A; B.$

# Operadors lògics

## Negació

```
neg(A) :- A, !, false.  
neg(_).
```

(Si A és cert, ! obliga a que neg(A) falli. Si és fals, la 2a def. té èxit.)

## Implicació

```
implies(A, B) :- A, !, B.  
implies(_, _).
```

(Semblant a la negació.)